

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2022/2023

MAY EXAMINATION

**UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND
ARCHITECTURE**

SATURDAY, 27 MAY 2023

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING
BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING

Instructions to Candidates:

This question paper consists of **TWO (2)** sections.

Section A consists of **THREE (3)** questions. **Answer ALL questions.**

Section B consists of **TWO (2)** questions. Choose **ONLY ONE (1)** question. Should a candidate answer more than **ONE (1)** question in Section B, marks will only be awarded for the **FIRST** question in the order the candidate submits the answers.

Each question carries 25 marks.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

SECTION A (ANSWER ALL questions)

Q1. (a) **List TWO (2) advantages of Boolean expression simplification.** (4 marks)

(b) A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. The output of the circuit has two conditions: HIGH and LOW. The output should be HIGH only if prime numbers are being input to the detector circuit.

(i) **Produce** a truth table for a 4-bit prime number detector circuit. (6 marks)

(ii) **Write** the maxterm notation from the truth table obtained in Q1(b)(i). (2 marks)

(c) **Sketch ONE** logic gate that can replace the logic circuit given in Figure Q1(c) by deriving the expression for each gate. (8 marks)

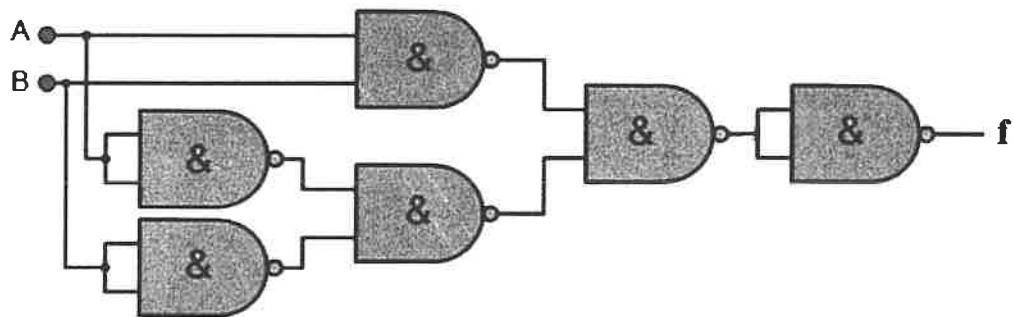


Figure Q1(c)

(d) **Sketch** a diagram to show the implementation of an 8:1 MUX using only 4:1 and 2:1 line MUX. (5 marks)

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Q2. (a) **List** FOUR (4) design principles of computer architecture. (4 marks)

(b) **Translate** the high-level code shown in Figure Q2(b) into MIPS assembly code by assuming all the variables are stored at saved register, \$s. (7 marks)

$$a = b - c;$$

$$f = (g + h) - (i + j);$$

Figure Q2(b)

(c) A MIPS assembly code is shown in Table Q2(c). **Show** the content of each register when each code is executed. Assume the initial value of all the registers are 0 (decimal). (14 marks)

Table Q2(c)

Assembly Code	Initial values					
	\$a0	\$s0	\$s1	\$t0	\$t1	\$t2
	0	0	0	0	0	0
addi \$a0, \$0, 10						
addi \$t1, \$0, 2000						
add \$s0, \$a0, \$0						
mult \$a0, \$s0						
mflo \$s1						
mul \$t0, \$s0, \$s1						
mul \$t2, \$t0, 3						

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Q3. (a) **List** THREE (3) differences between Reduced Instruction set Computer (RISC) and Complex Instruction Set Computers (CISC). (6 marks)

(b) Consider the following machine codes:

0x000D7100....(1)

0x35CFF100....(2)

(i) **Analyse** the machine code (1) and (2) to produce the appropriate MIPS assembly language. Show all the working steps. (13 marks)

(ii) Assume that register \$t5 holds the value of 0xFFFF0000. **Determine** the values of the other two registers when the instructions in Q3 (b)(i) is executed. Show your workings in binary and give the final answer in hexadecimal. (6 marks)

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

SECTION B (CHOOSE only ONE question)

Q4. (a) **List FIVE (5) major components of a computer system.** (5 marks)

(b) Briefly **describe** all types of pipeline hazard in a computer architecture. (9 marks)

(c) Two different compilers are used to execute a same program. Table Q4(c) shows the number of instruction and execution time. Assume the clock of the processor runs at 10 GHz.

Table Q4(c)

Compiler	No of instruction	Execution Time
Compiler X	10 billion	2.2s
Compiler Y	12 billion	1.9s

(i) **Compute** the average clock cycles per instruction (CPI) to run the program. (7 marks)

(ii) A new compiler (Compiler Z) is developed to execute the same program as mentioned in Q4(c). The compiler uses 11 billion instructions with an average CPI of 1.1. **Show** the compiler that has the highest performance by comparing the three compilers (compiler X, Y and Z).

(4 marks)

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

- Q5. (a) **Sketch** the steps required to translate a program from a high-level code to start executing the program. (6 marks)
- (b) **Illustrate** the instruction execution cycle in detail with the aid of a diagram. (8 marks)
- (c) The Universal Serial Bus (USB) greatly simplified adding peripherals by standardizing the cables and software configuration process.
- (i) **State** the encoding scheme used in USB, which improve the reliability of data transmission. (2 marks)
- (ii) A source data shown in Figure Q5 (c)(ii) is to be transmitted via USB. **Sketch** the waveform based on the encoding scheme stated in Q5(c)(i). (9 marks)

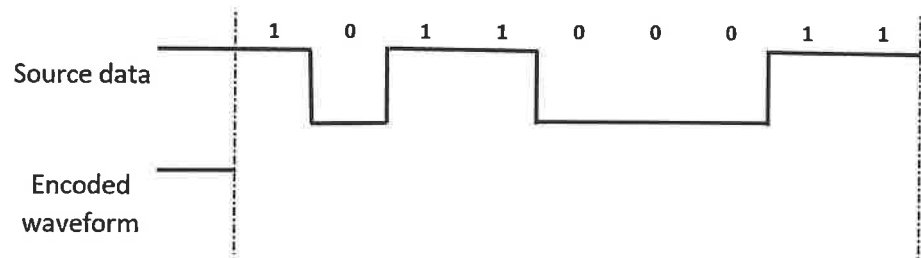


Figure Q5 (c)(ii)

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

APPENDIX A: MIPS Reference Data

MIPS Reference Data

CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	add R	$R[rd] = R[rs] + R[rt]$	(1) 0/20 _{hex}
Add Immediate	addi I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu I	$R[rt] = R[rs] + \text{SignExtImm}$	(2) 9 _{hex}
Add Unsigned	addu R	$R[rd] = R[rs] + R[rt]$	0/21 _{hex}
And	and R	$R[rd] = R[rs] \& R[rt]$	0/24 _{hex}
And Immediate	andi I	$R[rt] = R[rs] \& \text{ZeroExtImm}$	(3) 0 _{hex}
Branch On Equal	beq I	$\text{if}(R[rs] == R[rt])$ $PC = PC + 4 + \text{BranchAddr}$	(4) 4 _{hex}
Branch On Not Equal	bne I	$\text{if}(R[rs] != R[rt])$ $PC = PC + 4 + \text{BranchAddr}$	(4) 5 _{hex}
Jump	j J	$PC = \text{JumpAddr}$	(5) 2 _{hex}
Jump And Link	jal J	$R[31] = PC + 4; PC = \text{JumpAddr}$	(5) 3 _{hex}
Jump Register	jr R	$PC = R[rs]$	0/08 _{hex}
Load Byte Unsigned	lbu I	$R[rt] = (24'b0, M[R[rs]] + \text{SignExtImm})(7:0)$	(2) 24 _{hex}
Load Halfword Unsigned	lhu I	$R[rt] = (16'b0, M[R[rs]] + \text{SignExtImm})(15:0)$	(2) 25 _{hex}
Load Linked	ll I	$R[rt] = M[R[rs]] + \text{SignExtImm}$	(2,7) 30 _{hex}
Load Upper Imm.	lui I	$R[rt] = (\text{imm}, 16'b0)$	6 _{hex}
Load Word	lw I	$R[rt] = M[R[rs]] + \text{SignExtImm}$	(2) 23 _{hex}
Nor	nor R	$R[rd] = \sim(R[rs]) \& R[rt]$	0/27 _{hex}
Or	or R	$R[rd] = R[rs] R[rt]$	0/25 _{hex}
Or Immediate	ori I	$R[rt] = R[rs] \text{ZeroExtImm}$	(3) 4 _{hex}
Set Less Than	slt R	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$	0/26 _{hex}
Set Less Than Imm.	slti I	$R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$	2 _{hex}
Set Less Than Imm. Unsigned	sltiu I	$R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$	(2,6) 6 _{hex}
Set Less Than Unsig.	sltu R	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$	(6) 0/28 _{hex}
Shift Left Logical	sll R	$R[rd] = R[rs] \ll \text{shamt}$	0/00 _{hex}
Shift Right Logical	srl R	$R[rd] = R[rs] \gg \text{shamt}$	0/02 _{hex}
Store Byte	sb I	$M[R[rs] + \text{SignExtImm}](7:0) = R[rt](7:0)$	(2) 28 _{hex}
Store Conditional	sc I	$M[R[rs] + \text{SignExtImm}] = R[rt];$ $R[rt] = (\text{atomic}) ? 1 : 0$	(2,7) 38 _{hex}
Store Halfword	sh I	$M[R[rs] + \text{SignExtImm}](15:0) = R[rt](15:0)$	(2) 29 _{hex}
Store Word	sw I	$M[R[rs] + \text{SignExtImm}] = R[rt]$	(2) 2b _{hex}
Subtract	sub R	$R[rd] = R[rs] - R[rt]$	(1) 0/22 _{hex}
Subtract Unsigned	subu R	$R[rd] = R[rs] - R[rt]$	0/23 _{hex}

- (1) May cause overflow exception
 (2) SignExtImm = { 16(immediate[15]), immediate }
 (3) ZeroExtImm = { 16(1b'0), immediate }
 (4) BranchAddr = { 14(immediate[15]), immediate, 2'b0 }
 (5) JumpAddr = { PC+4[31:28], address, 2'b0 }
 (6) Operands considered unsigned numbers (vs. 2's comp.)
 (7) Atomic test/set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

R	opcode	rs	rt	rd	shamt	funct
31	26-25	21-20	16-15	11-10	6-5	0
I	opcode	rs	rt	immediate		
31	26-25	21-20	16-15	0		
J	opcode	address				
31	26-25	0				

ARITHMETIC CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION	OPCODE / FUNCT (Hex)
Branch On FP True	bclt FI	$\text{if}(\text{FPCond}) PC = PC + 4 + \text{BranchAddr}$	(4) 11/8/1-
Branch On FP False	bclt FI	$\text{if}(\text{FPCond}) PC = PC + 4 + \text{BranchAddr}$	(4) 11/8/0-
Divide	div R	$Lo = R[rs]/R[rt]; Hi = R[rs] \% R[rt]$	0/-/-/1a
Divide Unsigned	divu R	$Lo = R[rs]/R[rt]; Hi = R[rs] \% R[rt]$	(6) 0/-/-/1b
FP Add Single	add.s FR	$F[fd] = F[fs] + F[ft]$	11/10/-0
FP Add Double	add.d FR	$(F[fd], F[fd+1]) = (F[fs], F[fs+1]) + (F[ft], F[ft+1])$	11/11/-0
FP Compare Single	c.s.s* FR	$\text{FPCond} = (F[fs] \text{ op } F[ft]) ? 1 : 0$	11/10/-y
FP Compare Double	c.d.d* FR	$\text{FPCond} = ((F[fs], F[fs+1]) \text{ op } (F[ft], F[ft+1])) ? 1 : 0$ * (x is eq, lt, or le) (op is ==, <, or <=) (y is 32, 3c, or 3e)	11/11/-y
FP Divide Single	div.s FR	$F[fd] = F[fs] / F[ft]$	11/10/-3
FP Divide Double	div.d FR	$(F[fd], F[fd+1]) = (F[fs], F[fs+1]) / (F[ft], F[ft+1])$	11/11/-3
FP Multiply Single	mul.s FR	$F[fd] = F[fs] * F[ft]$	11/10/-2
FP Multiply Double	mul.d FR	$(F[fd], F[fd+1]) = (F[fs], F[fs+1]) * (F[ft], F[ft+1])$	11/11/-2
FP Subtract Single	sub.s FR	$F[fd] = F[fs] - F[ft]$	11/10/-1
FP Subtract Double	sub.d FR	$(F[fd], F[fd+1]) = (F[fs], F[fs+1]) - (F[ft], F[ft+1])$	11/11/-1
Load FP Single	lwc1 I	$F[rt] = M[R[rs] + \text{SignExtImm}]$	(2) 31/-/-/-
Load FP Double	lwc1 I	$F[rt+1] = M[R[rs] + \text{SignExtImm} + 4]$	(2) 35/-/-/-
Move From Hi	mthi R	$R[rd] = Hi$	0/-/-/10
Move From Lo	mthi R	$R[rd] = Lo$	0/-/-/12
Move From Control	mfc0 R	$R[rd] = CR[rs]$	10/0/-0
Multiply	mult R	$(Hi, Lo) = R[rs] * R[rt]$	0/-/-/18
Multiply Unsigned	multu R	$(Hi, Lo) = R[rs] * R[rt]$	(6) 0/-/-/19
Shift Right Arith.	sra R	$R[rd] = R[rs] \gg \text{shamt}$	0/-/-/3
Store FP Single	swc1 I	$M[R[rs] + \text{SignExtImm}] = F[rt]$	(2) 39/-/-/-
Store FP Double	swc1 I	$M[R[rs] + \text{SignExtImm}] = F[rt];$ $M[R[rs] + \text{SignExtImm} + 4] = F[rt+1]$	(2) 3d/-/-/-

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	funct	rs	fs	fd	funct
31	26-25	21-20	16-15	11-10	6-5	0
FI	opcode	funct	rs	immediate		
31	26-25	21-20	16-15			

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	$\text{if}(R[rs] < R[rt]) PC = \text{Label}$
Branch Greater Than	bgt	$\text{if}(R[rs] > R[rt]) PC = \text{Label}$
Branch Less Than or Equal	bltle	$\text{if}(R[rs] <= R[rt]) PC = \text{Label}$
Branch Greater Than or Equal	bgtle	$\text{if}(R[rs] >= R[rt]) PC = \text{Label}$
Load Immediate	li	$R[rd] = \text{immediate}$
Move	move	$R[rd] = R[rs]$

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$f8-\$f9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix A: MIPS Reference Data (continued)

③

OPCODES, BASE CONVERSION, ASCII SYMBOLS

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexa- decimal	ASCII Character	Decimal	Hexa- decimal	ASCII Character
(1)	all	add	00 0000	0	0	NUL	64	40	g
		sub	00 0001	1	1	SOH	65	41	A
	ori	mul	00 0010	2	2	STX	66	42	B
	jal	div	00 0011	3	3	ETX	67	43	C
	beq	sllv	00 0100	4	4	EOI	68	44	D
	bne	abs	00 0101	5	5	ENQ	69	45	E
	blez	orlv	00 0110	6	6	ACK	70	46	F
	bgtz	arav	00 0111	7	7	BEL	71	47	G
	addi	jr	00 1000	8	8	BS	72	48	H
	addiu	jalr	00 1001	9	9	HT	73	49	I
	slli	movz	00 1010	10	a	LF	74	4a	J
	slltu	movn	00 1011	11	b	VT	75	4b	K
	andi	syscall	00 1100	12	c	FF	76	4c	L
	ori	break	00 1101	13	d	CR	77	4d	M
	xori	ceil.w	00 1110	14	e	SO	78	4e	N
	lui	sync	00 1111	15	f	SI	79	4f	O
(2)	mfhi		01 0000	16	10	DLE	80	50	P
	mfhi		01 0001	17	11	DC1	81	51	Q
	mflo	movz	01 0010	18	12	DC2	82	52	R
	mflo	movn	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	^
			01 1110	30	1e	RS	94	5e	_
			01 1111	31	1f	US	95	5f	`
	lb	add	10 0000	32	20	Space	96	60	~
	lh	addu	10 0001	33	21	!	97	61	a
	lwl	sub	10 0010	34	22	"	98	62	b
	lw	subu	10 0011	35	23	#	99	63	c
	lbu	and	10 0100	36	24	\$	100	64	d
	lbu	or	10 0101	37	25	%	101	65	e
	lwr	xor	10 0110	38	26	&	102	66	f
	sw	nor	10 0111	39	27	'	103	67	g
	sb		10 1000	40	28	(	104	68	h
	sh		10 1001	41	29	)	105	69	i
	sll		10 1010	42	2a	*	106	6a	j
	slltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
			10 1110	46	2e	.	110	6e	n
			10 1111	47	2f	/	111	6f	o
	swr		11 0000	48	30	0	112	70	p
	coache		11 0001	49	31	1	113	71	q
	l1	lpc	11 0010	50	32	2	114	72	r
	lwc2	lpc	11 0011	51	33	3	115	73	s
	prel	lpc	11 0100	52	34	4	116	74	t
	lde1	lpc	11 0101	53	35	5	117	75	u
	lde2	lpc	11 0110	54	36	6	118	76	v
		lpc	11 0111	55	37	7	119	77	w
	se	lpc	11 1000	56	38	8	120	78	x
	swe1	lpc	11 1001	57	39	9	121	79	y
	swe2	lpc	11 1010	58	3a	:	122	7a	z
		lpc	11 1011	59	3b	:	123	7b	{
	sdcl	lpc	11 1100	60	3c	<	124	7c	}
	sdcl	lpc	11 1101	61	3d	=	125	7d	
		lpc	11 1110	62	3e	>	126	7e	~
		lpc	11 1111	63	3f	?	127	7f	DEL

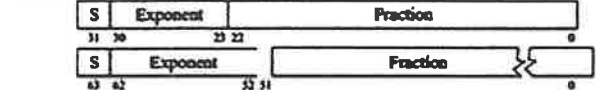
(1) opcode(31:26) = 0
 (2) opcode(31:26) = 17_{hex}; if funt(25:21) = 16_{hex} (10_{hex}) f = s (single);
 if funt(25:21) = 17_{hex} (11_{hex}) f = d (double)

IEEE 754 FLOATING-POINT STANDARD

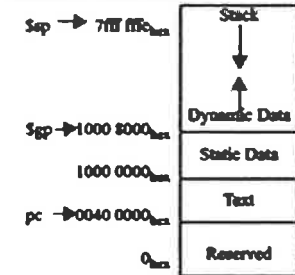
$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

where Single Precision Bias = 127,
 Double Precision Bias = 1023.

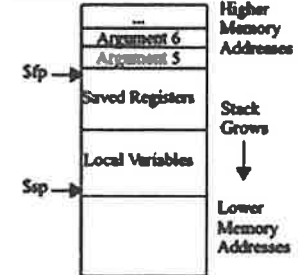
IEEE Single Precision and
 Double Precision Formats:



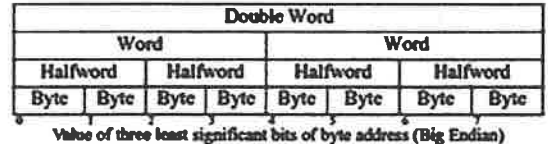
MEMORY ALLOCATION



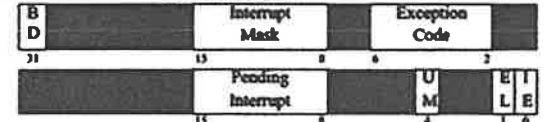
STACK FRAME



DATA ALIGNMENT



EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS



BD - Branch Delay, UM - User Mode, EL - Exception Level, IE - Interrupt Enable
 EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RJ	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10³ for Disk, Communication; 2³ for Memory)

SIZE	PRE- FIX	SIZE	PRE- FIX	SIZE	PRE- FIX	SIZE	PRE- FIX
10 ³ , 2 ¹⁰	Kilo-	10 ¹⁵ , 2 ⁵⁰	Peta-	10 ⁻³	milli-	10 ⁻¹⁵	femto-
10 ⁶ , 2 ²⁰	Mega-	10 ¹⁸ , 2 ⁶⁰	Eta-	10 ⁻⁶	micro-	10 ⁻¹⁸	atto-
10 ⁹ , 2 ³⁰	Giga-	10 ²¹ , 2 ⁷⁰	Zetta-	10 ⁻⁹	nano-	10 ⁻²¹	zepto-
10 ¹² , 2 ⁴⁰	Tera-	10 ²⁴ , 2 ⁸⁰	Yotta-	10 ⁻¹²	pico-	10 ⁻²⁴	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix B: MIPS Instruction Syntax

Instruction	Syntax	Operation
add	add rd, rs, rt	$[rd] = [rs] + [rt]$
add immediate	addi rt, rs, imm	$[rt] = [rs] + \text{SignImm}$
add immediate unsigned	addiu rt, rs, imm	$[rt] = [rs] + \text{SignImm}$
add unsigned	addu rd, rs, rt	$[rd] = [rs] + [rt]$
and	and rd, rs, rt	$[rd] = [rs] \& [rt]$
and immediate	andi rt, rs, imm	$[rt] = [rs] \& \text{ZeroImm}$
branch if equal	beq rs, rt, label	if $([rs] == [rt])$ PC = BTA
branch if not equal	bne rs, rt, label	if $([rs] != [rt])$ PC = BTA
divide	div rs, rt	$[lo] = [rs]/[rt], [hi] = [rs]\%[rt]$
divide unsigned	divu rs, rt	$[lo] = [rs]/[rt], [hi] = [rs]\%[rt]$
jump	j label	PC = JTA
jump and link	jal label	$\$ra = PC + 4$, PC = JTA
jump register	jrrs	PC = [rs]
load byte	lb rt, imm(rs)	$[rt] = \text{SignExt}([Address]7:0)$
load byte unsigned	lbu rt, imm(rs)	$[rt] = \text{ZeroExt}([Address]7:0)$
load upper immediate	lui rt, imm	$[rt] = \{imm, 16'b0\}$
load word	lw rt, imm(rs)	$[rt] = [Address]$
move from hi	mfhi rd	$[rd] = [hi]$
move from lo	mflo rd	$[rd] = [lo]$
move from/to coprocessor 0	mfc0 rt, rd /mtc0 rt, rd	$[rt] = [rd]/[rd] = [rt]$
move to hi	mthi rs	$[hi] = [rs]$
move to lo	mtlo rs	$[lo] = [rs]$
multiply	mult rs, rt	$\{[hi], [lo]\} = [rs] \times [rt]$
multiply (32-bit result)	mul rd, rs, rt	$[rd] = [rs] \times [rt]$
multiply unsigned	multu rs, rt	$\{[hi], [lo]\} = [rs] \times [rt]$
nor	nor rd, rs, rt	$[rd] = \sim([rs] \mid [rt])$
or	or rd, rs, rt	$[rd] = [rs] \mid [rt]$
or immediate	ori rt, rs, imm	$[rt] = [rs] \mid \text{ZeroImm}$
set less than	slt rd, rs, rt	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$
set less than immediate	slti rt, rs, imm	$[rs] < \text{SignImm} ? [rt] = 1 : [rt] = 0$
set less than immediate unsigned	sltiu rt, rs, imm	$[rs] < \text{SignImm} ? [rt] = 1 : [rt] = 0$
set less than unsigned	sltu rd, rs, rt	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$
shift left logical	sll rd, rt, shamt	$[rd] = [rt] \ll \text{shamt}$
shift right arithmetic	sra rd, rt, shamt	$[rd] = [rt] \ggg \text{shamt}$
shift right logical	srl rd, rt, shamt	$[rd] = [rt] \gg \text{shamt}$
store byte	sb rt, imm(rs)	$[Address]7:0 = [rt]7:0$
store word	sw rt, imm(rs)	$[Address] = [rt]$
subtract	sub rd, rs, rt	$[rd] = [rs] - [rt]$
subtract unsigned	subu rd, rs, rt	$[rd] = [rs] - [rt]$
xor	xor rd, rs, rt	$[rd] = [rs] \wedge [rt]$
xor immediate	xori rt, rs, imm	$[rt] = [rs] \wedge \text{ZeroImm}$

